STUDENT PERFORMANCE ANALYSIS SYSTEM

PROJECT DOCUMENTATION

Submitted by

Name: _____

Roll Number: _____

Department: _____

College Name: _____

Academic Year: 2026–2027

Technologies: Python • Streamlit • MySQL • Pandas • NumPy

Acknowledgement

I would like to express my sincere gratitude to my project guide and faculty members for their support and guidance. I also thank my institution for providing the opportunity to develop this project. This project helped me understand how Python, Streamlit, and MySQL can be combined to build a data-driven application.

Abstract

The Student Performance Analysis System is a web-based application designed to manage student details, subject information, assessment marks, and academic performance reports. The application uses Python and Streamlit for application logic and the user interface, while MySQL stores student, subject, and marks records. Pandas and NumPy support data processing and numerical analysis. The system provides modules for student management, subject management, marks entry and maintenance, and reports. It calculates percentages and grades, summarizes subject and section performance, identifies top performers and students below a selected threshold, and supports CSV export of report data.

Table of Contents

- Chapter 1 — Introduction
- Chapter 2 — System Analysis
- Chapter 3 — System Design
- Chapter 4 — Implementation
- Chapter 5 — Testing and Results
- Chapter 6 — Conclusion and Future Scope
- References

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Student Performance Analysis System is a web-based application that helps organize academic records and generate performance summaries. It provides a single interface for maintaining student information, subject information, and assessment marks. The stored data is used to calculate performance indicators and present them through tables, charts, and downloadable reports.

1.2 Problem Statement

Managing student marks manually can require repeated calculations and make it difficult to prepare consistent reports. When records are maintained in separate files, it can also take more time to identify students who may need academic support. This project provides a centralized system for recording assessment information and generating summaries from the stored data.

1.3 Objectives

- Maintain student records including roll number, name, and section.
- Maintain subject information including maximum marks and pass marks.
- Record, update, and remove assessment marks.
- Calculate percentages, grades, and pass/fail status.
- Generate student-wise, subject-wise, and section-wise summaries.
- Identify top performers and students below a selected performance threshold.
- Allow report data to be downloaded as a CSV file.

1.4 Scope

The current scope covers student and subject record management, assessment marks management, performance calculations, report tables, charts, and CSV export. Features such as user login, role-based permissions, attendance tracking, and PDF report generation are possible future enhancements rather than confirmed current features.

CHAPTER 2: SYSTEM ANALYSIS

2.1 Existing System

In a manual or spreadsheet-based process, student details and assessment marks may be stored separately. Percentages, grades, and summaries may need to be calculated repeatedly, and report preparation can become time-consuming as the amount of data increases.

2.2 Proposed System

The proposed application provides a centralized interface connected to a MySQL database. Users can manage students, subjects, and marks through Streamlit pages. Python logic validates input and prepares summaries, while Pandas and NumPy support tabular and numerical analysis.

2.3 Advantages

- Centralized storage of academic records.
- Consistent calculations for percentages and performance summaries.
- Convenient updating and maintenance of records.
- Clear tables and charts for reviewing performance.
- CSV export for additional analysis in spreadsheet software.

2.4 Functional Requirements

- The system should allow student records to be added, edited, and deleted.
- The system should allow subjects and their mark limits to be maintained.
- The system should allow assessment marks to be added, updated, and removed.
- The system should prevent duplicate student-subject-assessment combinations.
- The system should display reports and allow CSV download.

2.5 Non-Functional Requirements

- Usability: the interface should be understandable to users with basic computer skills.
- Data integrity: database keys and constraints should help prevent duplicate or invalid records.
- Maintainability: code is separated into application, database, and analytics files.
- Reliability: database connectivity and validation should be available when the application is used.

CHAPTER 3: SYSTEM DESIGN

3.1 System Architecture

The user interacts with the Streamlit interface. The Python application handles page actions and validation, queries MySQL through the database module, and uses Pandas and NumPy to prepare analytics. Results are displayed as tables, charts, and downloadable CSV data.

User	Streamlit UI	Python logic Pandas / NumPy	MySQL database

Flow: User → Streamlit pages → Python/database and analytics logic → MySQL records → results displayed in the interface.

3.2 Database Design

The database is named student_performance by default. The main tables created by database.py are:

Table	Important fields	Purpose
students	student_id (PK), roll_no (unique), name, section	Stores student details.
subjects	subject_id (PK), subject_name (unique), max_marks, pass_marks	Stores subject and mark limits.
marks	mark_id (PK), student_id (FK), subject_id (FK), assessment, marks_obtained	Stores assessment marks and links students to subjects.

Database rules: roll_no and subject_name are unique. The marks table has foreign keys to students and subjects. A unique key on (student_id, subject_id, assessment) prevents the same student from having duplicate records for the same subject and assessment name.

3.3 Data Validation

- Roll numbers must be unique.
- Subject maximum marks must be positive, and pass marks must be between zero and maximum marks.
- Marks entry in the application is limited to the subject's valid mark range.
- Duplicate student, subject, and assessment combinations are rejected by the database.

3.4 Main Project Files

File	Responsibility
app.py	Streamlit pages, forms, data display, and user interactions.
database.py	Database connection, table initialization, and query helpers.
analytics.py	Grade calculation and student, subject, and section summaries.
requirements.txt	List of Python packages required by the project.

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment

The project is organized in Visual Studio Code. Python is used to implement the application. Streamlit provides the browser-based interface, MySQL stores persistent data, and Pandas and NumPy support analytics.

4.2 Dashboard Module

The Dashboard provides an overview of available student-performance information. Its exact displayed metrics depend on the records stored in the database.

4.3 Student Management Module

The Students page supports adding, editing, and deleting student records. Each student has a unique roll number, name, and section.

4.4 Subject Management Module

The Subjects page maintains subject names, maximum marks, and pass marks. Validation ensures the pass mark does not exceed the maximum mark.

4.5 Marks Management Module

The Marks page supports adding assessment marks and updating or removing existing records. Each record links a student and a subject with an assessment name. The database prevents duplicate combinations of student, subject, and assessment.

4.6 Reports and Analytics Module

The Reports page displays student performance summaries, top performers, students below a selected threshold, subject summaries, and section summaries. It also provides charts and a CSV download option.

4.7 Grade and Status Calculation

Percentage	Grade
90% and above	A
80% to below 90%	B
70% to below 80%	C
60% to below 70%	D
Below 60%	F

Student percentage is calculated from total marks obtained divided by total possible marks across recorded assessments, multiplied by 100. The overall status is Pass only if every recorded assessment meets the pass mark for its subject; otherwise it is Fail.

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Approach

Testing should confirm that forms accept valid values, reject invalid or duplicate data, and that report pages display the expected summaries. The following are suggested test cases; update the result column after executing each test in your own application.

Test ID	Test action	Expected result	Actual result
TC-01	Add a student with a new roll number	Student record is saved	Fill after testing
TC-02	Add an existing roll number	Duplicate is rejected	Fill after testing
TC-03	Enter marks within valid range	Assessment is saved	Fill after testing
TC-04	Repeat same student + subject + assessment	Duplicate is rejected	Fill after testing
TC-05	Update marks without creating a duplicate	Record is updated	Fill after testing
TC-06	Open Reports page	Summaries and charts display	Fill after testing
TC-07	Download CSV report	CSV file downloads	Fill after testing

5.2 Application Screenshots

Insert screenshots of the running application in this section. Recommended screenshots: Dashboard, Students page, Subjects page, Marks page, and Reports page. Add a short caption below each image explaining the page and what it demonstrates.

5.3 Results

When the database contains valid records, the application can display student-level and subject-level performance summaries, identify records below a chosen threshold, and provide report data for CSV download. Actual screenshots and test outcomes should be included after verification.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Student Performance Analysis System combines student and subject record management with assessment tracking and academic analytics. The use of Streamlit makes the system accessible through a browser, while MySQL provides persistent data storage. Python, Pandas, and NumPy support data validation and performance summaries. The application can help present academic information in a structured and consistent format.

6.2 Limitations and Considerations

- The accuracy of reports depends on correct and complete data entry.
- The current database rules restrict duplicate student-subject-assessment combinations.
- Access control, deployment security, and backup procedures should be reviewed before using the system with real institutional data.
- Average raw marks may not be directly comparable across subjects with different maximum marks; percentage is the normalized measure.

6.3 Future Enhancements

- Add secure login and role-based access for administrators and teachers.
- Generate downloadable PDF reports.
- Add attendance and term-wise historical performance tracking.
- Provide automated database backups and recovery procedures.
- Add more configurable charts and trend analysis.

REFERENCES

- Python documentation: https://docs.python.org/3/
- Streamlit documentation: https://docs.streamlit.io/
- MySQL documentation: https://dev.mysql.com/doc/
- Pandas documentation: https://pandas.pydata.org/docs/
- NumPy documentation: https://numpy.org/doc/

— End of Documentation —